FIAP

GRADUAÇÃO

45697056

ES
Desenvolvimento Mobile

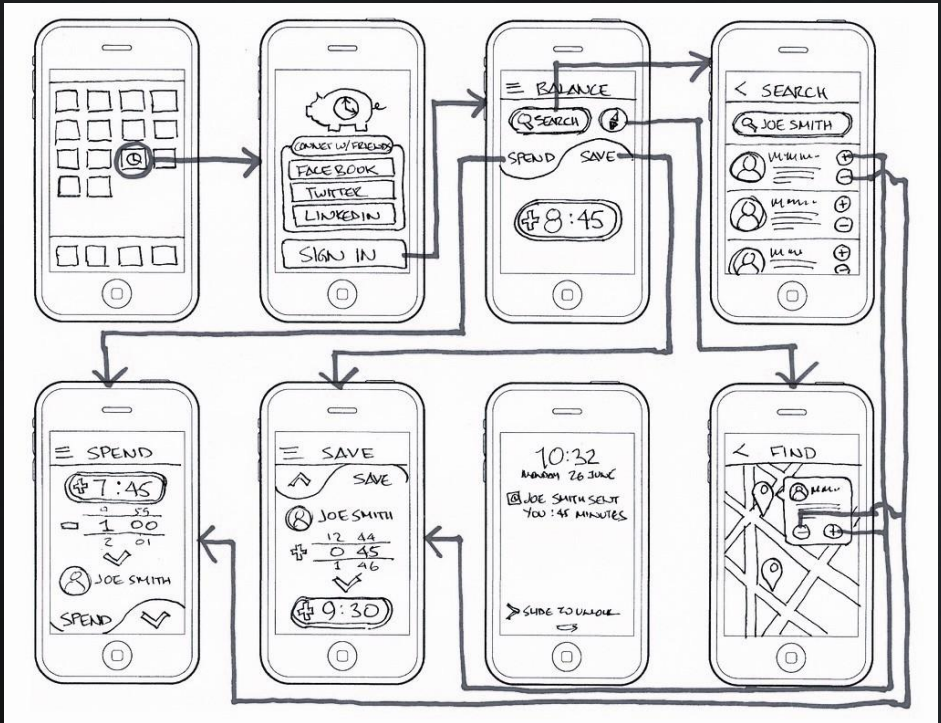
Prof. Adeilton Meneses

profadeilton.meneses@fiap.com.br

Navegação

45697056

Até agora nós aprendemos diversos componentes e práticas de React Native. Mas algo básico em aplicativos, ainda não exploramos, que é a navegação.



Navegação

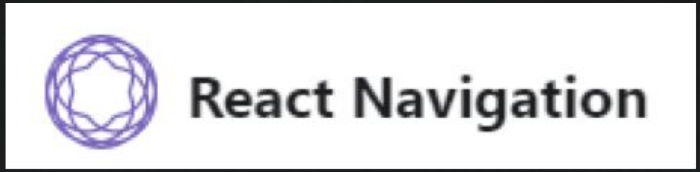
45697056

Para a navegação, poderíamos utilizar uma estratégia similar ao HTML puro, mas já existem diversas bibliotecas que fornecem para nós uma alternativa mais simples e mais desenvolvida. No nosso caso utilizaremos o **React-navigation**

Documentação: https://reactnavigation.org/docs/getting-started

Navegação

45697056




```
npm install @react-navigation/native
```

```
npm install react-native-screens react-native-safe-area-context
```

```
npm install @react-navigation/native-stack
```

Navegação

45697056

- **Stack Navigator:** Cria uma pilha de telas. É ideal para fluxos sequenciais, como uma lista de produtos que leva a uma tela de detalhes, e a partir dela, a uma tela de carrinho. A transição é geralmente horizontal.

- **Tab Navigator:** Cria uma barra de abas (tabs), geralmente na parte inferior da tela, permitindo que o usuário alterne rapidamente entre as seções principais do aplicativo (ex: Home, Pesquisar, Perfil).

- **Drawer Navigator:** Cria um menu lateral (gaveta) que desliza para revelar as opções de navegação. É comumente usado para funcionalidades secundárias ou menos prioritárias.

Métodos de Navegação

45697056

Principais métodos do objeto **navigation**:

- **navigation.navigate('NomeDaTela')**: Navegar para uma nova tela.
- **navigation.goBack()**: Voltar para a tela anterior.
- **navigation.push('NomeDaTela')**: Empilhar uma nova instância da mesma tela
- **navigation.replace('NomeDaTela')**: Substituir a tela atual por uma nova.

Passando Dados entre Telas: Mostre como enviar parâmetros (**params**) para a tela de destino e como acessá-los (**route.params**).

Header

45697056

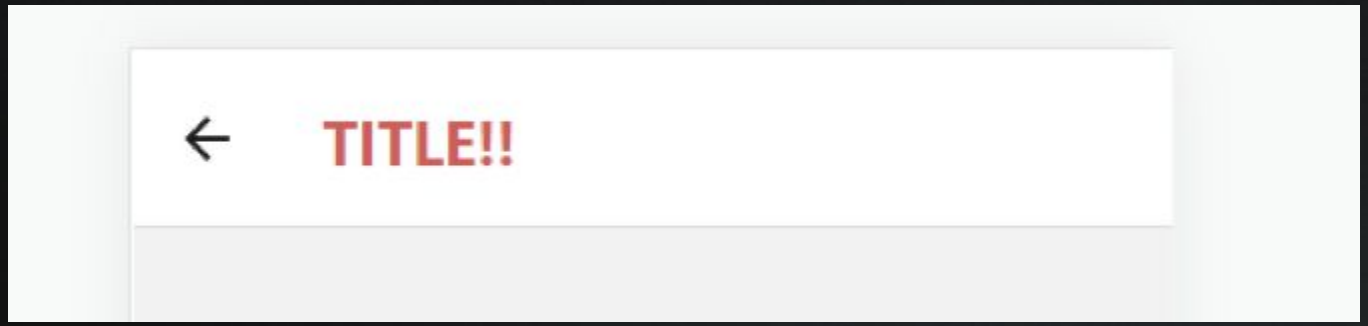
O Header é fornecido de maneira simples para nós, mas podemos customizá-lo ou escondê-lo se assim quisermos
Para isso, usaremos uma Prop especial headerStyle, que pode ser usada na screen, ou no Navigator, afetando todas as Screens

```
<Stack.Screen
  name="Home"
  component={HomeScreen}
  options={{ title: 'My home' }}
/>
```

Header

45697056

O **Title** e o **BackButton** podem ser substituídos por componentes personalizados, assim, nosso header é bem flexível
É possível também criar um botão adicional na direita, se assim quisermos



Navegação

45697056

```
const Stack = createNativeStackNavigator();

function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen name="Home" component={HomeScreen} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

Navegação

45697056

Container

O Container é o nosso Provider. Ele deve encapsular todos os componentes da navegação. Ele é o responsável pelo estado da navegação.
Raramente usaremos ele com alguma prop, ou precisaremos de sua referência. O caso mais comum será passar um initialState, para modificarmos a tela inicial do nosso app em casos especiais, como persistência de estado

Navegação

45697056

Navigator

O Navigator é responsável por definir o tipo de navegação que teremos. Temos a opção de drawer, tab ou stack (o mais comum). Ele decide como nossas telas serão renderizadas.
Podemos ter múltiplos Navigators, inclusive cascateados (nested navigation). Iremos adentrar mais afundo no comportamento deles neste caso.

Navegação

45697056

Screen

Aqui é onde nós definimos a nossa tela de fato. É obrigatório passarmos um name (único) e um component, que é a página que iremos renderizar ao acessar essa rota específica.

Navegação

45697056

Como navegar entre telas?

Nossa tela inicial será a primeira tela declarada, ou a tela passada na prop initialRouteName ao nosso navigator.

Cada tela declarada terá acesso a uma prop especial, o **navigation**, que nos permite fazer o controle do fluxo de telas.

Vamos conhecer o código?

Criar projeto

45697056

npx create-expo-app - -template

- Cria um novo projeto Expo com o nome **projeto-navigator-initial** usando o template blank ou de sua preferência.
- npx: executa pacotes sem precisar instalá-los globalmente.
- create-expo-app: ferramenta para iniciar projetos Expo.
- –template: lista templates (com React Native + Expo).

Navegação

45697056

Stack: Pilha de telas (como a navegação de um navegador web).

```
const Stack = createNativeStackNavigator();

function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator initialRouteName="Home">
        <Stack.Screen name="Home" component={HomeScreen} />
        <Stack.Screen name="Details" component={DetailsScreen} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

Instalar bibliotecas de navegação - Stack

45697056

- **npm install @react-navigation/native @react-navigation/stack react-native-screens react-native-safe-area-context react-native-gesture-handler**

@react-navigation/native

Este pacote é o **coração** do React Navigation. Ele contém a **lógica central** de navegação e as APIs base que permitem a transição entre telas, gerenciamento de rotas e o estado geral de navegação do seu aplicativo.

@react-navigation/stack

Este pacote é um **tipo de navegador** específico que cria uma experiência de navegação em **pilha**. É como um histórico de páginas da web: quando você navega para uma nova tela, ela é colocada no topo da pilha. Para voltar, você retira a tela do topo. É o tipo de navegação mais comum para aplicativos que usam botões de "voltar".

Instalar bibliotecas de navegação - Stack

45697056

- **npm install @react-navigation/native @react-navigation/stack react-native-screens react-native-safe-area-context react-native-gesture-handler**

react-native-screens

Este pacote otimiza o uso de memória e a performance de aplicativos que usam múltiplas telas. Ele **expõe a API nativa** de telas do iOS e Android para o React Native, permitindo que o sistema operacional lide de forma mais eficiente com as telas que não estão visíveis, liberando recursos e melhorando a fluidez das transições.

react-native-safe-area-context

Esse pacote ajuda a lidar com as áreas seguras do dispositivo, como a **barra de status, a barra de navegação e as "ilhas" ou "notches"** de iPhones e Androids modernos. Ele garante que o conteúdo da sua tela não fique escondido sob essas áreas, mantendo a interface visível e usável.

Instalar bibliotecas de navegação - Stack

45697056

- **npm install @react-navigation/native @react-navigation/stack react-native-screens react-native-safe-area-context react-native-gesture-handler**

react-native-gesture-handler

Esta biblioteca é fundamental para o bom funcionamento das **transições e animações** de navegação. Ela fornece um sistema nativo de reconhecimento de gestos, como o deslizar de dedo, que são essenciais para a experiência de navegação, como o gesto de "voltar" em iOS ou o gesto de deslizar para abrir uma gaveta de navegação.

45697056

Projeto Stack Navigator

https://github.com/admeneses/stack-navigator-initial

Como executar a primeira vez:

- clonar o projeto
- npm i

Depois de configurado:

npm start + (a plataforma que será executado)

Navegação

45697056

Tabs: Abas na parte inferior da tela.

```
const TabGroup = () => {
    return (
        <Tab.Navigator screenOptions={{ headerShown: true, header: TabHeader }}>
            <Tab.Screen name="Tab1" component={Tab1} />
            <Tab.Screen name="Tab2" component={Tab2} />
            <Tab.Screen name="Tab3" component={Tab3} />
        </Tab.Navigator>
    )
}
```

Instalar bibliotecas de navegação - Tabs

45697056

- **npm install @react-navigation/native @react-navigation/bottom-tabs @react-navigation/stack react-native-screens react-native-safe-area-context**

@react-navigation/bottom-tabs

Este pacote cria um tipo de navegador com **abas na parte inferior da tela**. É ideal para aplicativos que têm poucas seções principais, permitindo que o usuário alterne rapidamente entre elas com um toque.

45697056

Projeto Tabs Navigator

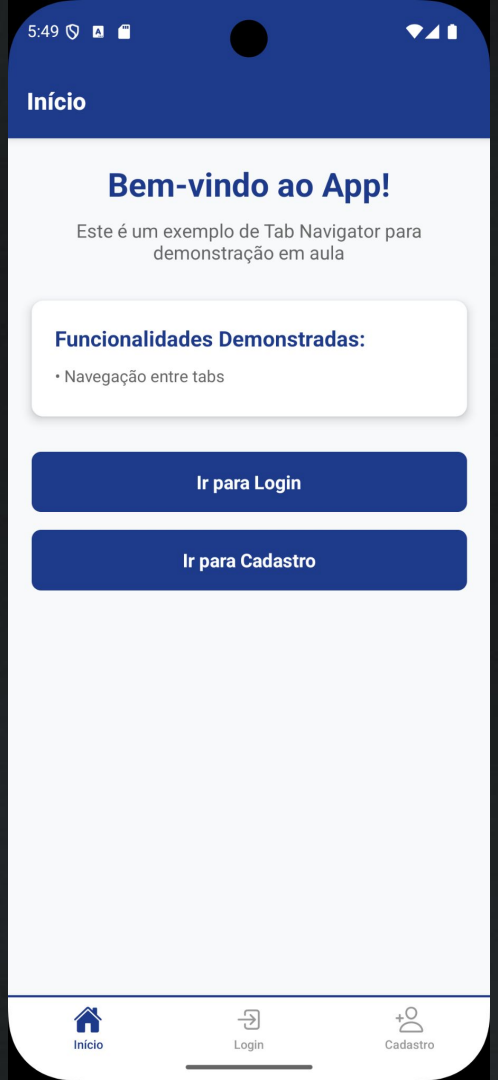
https://github.com/admeneses/tabs-navigator-initial

Como executar a primeira vez:

- clonar o projeto
- npm i

Depois de configurado:

npm start + (a plataforma que será executado)



Navegação

Drawer: Menu lateral.

```
<Drawer.Navigator
  drawerContent={(props) => (
    <CustomDrawerContent {...props} userEmail={userEmail} />
  )}
  screenOptions={{ …
  }}
>
  <Drawer.Screen
    name="Home"
    component={HomeScreen}
    options={{
      drawerLabel: 'Início',
      title: titleApp,
    }}
  />
  <Drawer.Screen
    name="Profile"
    component={ProfileScreen}
    options={{
      drawerLabel: 'Perfil',
      title: titleApp,
    }}
  />
  <Drawer.Screen
    name="Settings"
    component={SettingsScreen}
    options={{
      drawerLabel: 'Configurações',
      title: titleApp,
    }}
  />
</Drawer.Navigator>
```

Instalar bibliotecas de navegação - Drawer

- **npm install @react-navigation/native @react-navigation/drawer @react-navigation/stack react-native-screens react-native-safe-area-context react-native-gesture-handler react-native-reanimated**

@react-navigation/drawer

Este pacote cria um navegador do tipo **"gaveta" (drawer)**. Ele é ideal para menus laterais que podem ser abertos deslizando o dedo da borda da tela ou tocando em um ícone de menu.

react-native-reanimated

Esta biblioteca é usada para criar **animações e interações fluidas e de alto desempenho** na interface do usuário. Ela é uma dependência fundamental para que as animações de navegação do React Navigation, como as transições do `drawer`, funcione suavemente.

Projeto Drawer Navigator

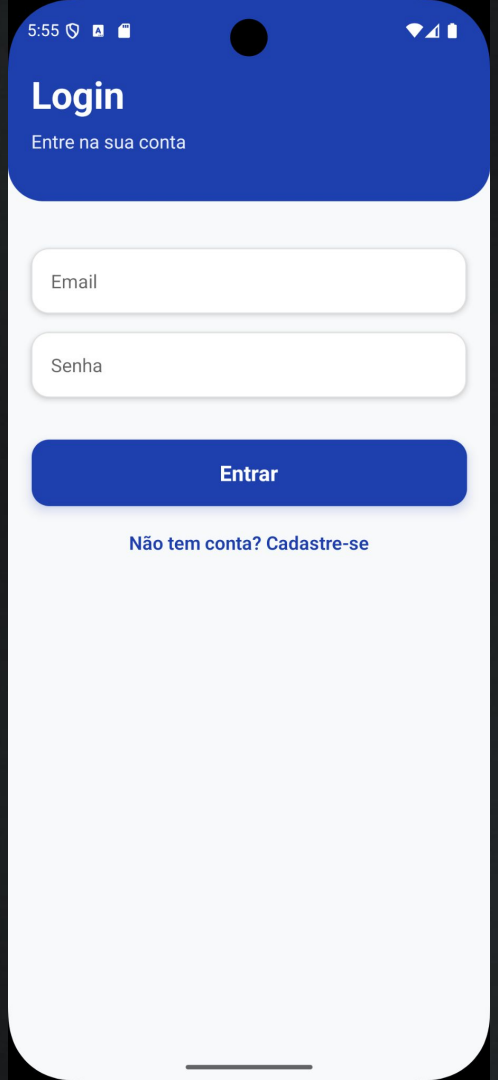
https://github.com/admeneses/drawer-navigator-initial

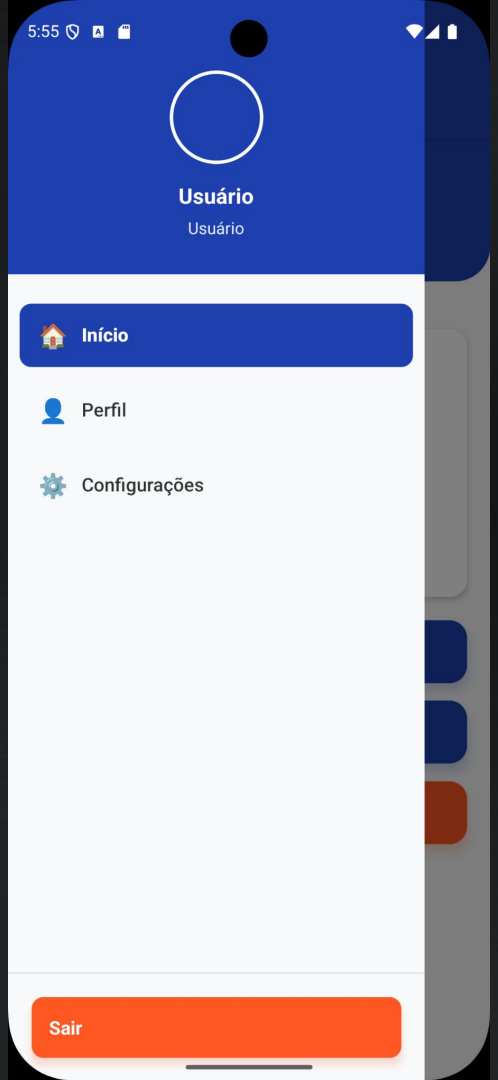
Como executar a primeira vez:

- clonar o projeto
- npm i

Depois de configurado:

npm start + (a plataforma que será executado).





Obrigado

2025-Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proíbido sem o consentimento formal, por escrito, do Professor Wellington Cidade Silva Tenorio.